

Experiment 10 — SonarQube Static Code Analysis Lab

Objective

To understand and perform static code analysis using SonarQube by:

- Running a SonarQube server using Docker
 - Scanning a Java project using Sonar Scanner (via Maven plugin)
 - Viewing detected bugs, vulnerabilities, and code smells in the dashboard
-

Theory

Problem Statement

Code bugs and security vulnerabilities are often discovered too late in the development cycle. Manual reviews are slow and hard to scale.

Static code analysis tools like **SonarQube** automatically analyze source code without executing it.

What is SonarQube?

SonarQube is an open-source platform used for **continuous inspection of code quality**.

It detects:

- Bugs
 - Security vulnerabilities
 - Code smells
 - Code duplication
 - Technical debt
 - Test coverage
-

SonarQube Architecture

SonarQube has two main components.

SonarQube Server

Responsible for:

- storing results
- applying analysis rules
- enforcing quality gates
- providing a dashboard

Runs at:

```
http://localhost:9000
```

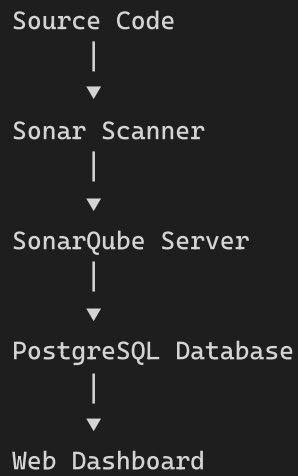
Sonar Scanner

Scanner:

- reads source code
- gathers project metadata
- sends results to the server

The **server performs the real analysis**, not the scanner.

Architecture Flow



Step 1 — Start SonarQube Server

Create `docker-compose.yml`

```

GNU nano 7.2                                     docker-compose.yml *
version: '3.8'

services:
  # --- Database ---
  sonar-db:
    image: postgres:13
    container_name: sonar-db
    environment:
      POSTGRES_USER: sonar          # DB username
      POSTGRES_PASSWORD: sonar      # DB password
      POSTGRES_DB: sonarqube        # DB name
      POSTGRES_HOST_AUTH_METHOD: trust # allow connections without password (for simplicity in this lab)
    volumes:
      - sonar-db-data:/var/lib/postgresql/data # persist data across restarts
    networks:
      - sonarqube-lab

  # --- SonarQube Server ---
  sonarqube:
    image: sonarqube:lts-community
    container_name: sonarqube
    ports:
      - "9000:9000" # access dashboard at http://localhost:9000
    environment:
      SONAR_JDBC_URL: jdbc:postgresql://sonar-db:5432/sonarqube
      SONAR_JDBC_USERNAME: sonar
      SONAR_JDBC_PASSWORD: sonar
    volumes:
      - sonar-data:/opt/sonarqube/data
      - sonar-extensions:/opt/sonarqube/extensions
    depends_on:
      - sonar-db # wait for DB to start first
    networks:
      - sonarqube-lab

# Named volumes (Docker manages storage location)
volumes:
  sonar-db-data:
  sonar-data:
  sonar-extensions:

# Isolated network so containers can talk to each other by name
networks:
  sonarqube-lab:
    driver: bridge

```

Start containers:

```

yuvraj@lucif3r: ~/lab9 $ docker compose up -d
WARN[0000] /home/yuvraj_singh/Lab9/docker compose yml: the attribute version is obsolete it will be ignored, please remove it to avoid pote
ntial confusion
[+] up 34/34
✓Image sonarqube:lts-community Pulled 131.3s
✓Image postgres:13 Pulled 111.7s
✓Network lab9_sonarqube-lab Created 0.0s
✓Volume lab9_sonar-data Created 0.0s
✓Volume lab9_sonar-extensions Created 0.0s
✓Volume lab9_sonar-db-data Created 0.0s
✓Container sonar-db Started 0.5s
✓Container sonarqube Started 0.4s

```

Check logs:

```

yuvraj_singh@lucif3r: ~/lab9 $ docker compose logs -f sonarqube
WARN[0000] /home/yuvraj_singh/Lab9/docker compose .yml: the attribute version is obsolete it will be ignored, please remove it to avoid pote
ntial confusion
sonarqube | 2026.04.29 17:28:16 INFO app[[o.s.a.AppFileSystem] Cleaning or creating temp directory /opt/sonarqube/temp
sonarqube | 2026.04.29 17:28:16 INFO app[[o.s.a.es.EsSettings] Elasticsearch listening on [HTTP: 127.0.0.1:9001, TCP: 127.0.0.1:34159]
sonarqube | 2026.04.29 17:28:16 INFO app[[o.s.a.ProcessLauncherImpl] Launch process[ELASTICSEARCH] from [/opt/sonarqube/elasticsearch]: /opt/
sonarqube/elasticsearch/bin/elasticsearch
sonarqube | 2026.04.29 17:28:16 INFO app[[o.s.a.SchedulerImpl] Waiting for Elasticsearch to be up and running
sonarqube | 2026.04.29 17:28:17 INFO es[[o.e.n.Node] version[7.17.15], pid[29], build[default/tar/0b8ecfb4378335f4689c4223d1f1115f16bef3ba/20
23-11-10T22:03:46.987399016Z], OS[Linux/6.6.87.2-microsoft-standard-WSL2/amd64], JVM[Eclipse Adoptium/OpenJDK 64-Bit Server VM/17.0.16/17.0.16+8
]
sonarqube | 2026.04.29 17:28:17 INFO es[[o.e.n.Node] JVM home [/opt/java/openjdk]
sonarqube | 2026.04.29 17:28:17 INFO es[[o.e.n.Node] JVM arguments [-XX:+UseG1GC, -Djava.io.tmpdir=/opt/sonarqube/temp, -XX:ErrorFile=/opt/so
narqube/logs/es_hs_err_pid%p.log, -Des.networkaddress.cache.ttl=60, -Des.networkaddress.cache.negative.ttl=10, -XX:+AlwaysPreTouch, -Xss1m, -Dja
va.awt.headless=true, -Dfile.encoding=UTF-8, -Djna.nosys=true, -Djna.tmpdir=/opt/sonarqube/temp, -XX:-OmitStackTraceInFastThrow, -Dio.netty.noUn
safe=true, -Dio.netty.noKeySetOptimization=true, -Dio.netty.recycler.maxCapacityPerThread=0, -Dio.netty allocator.numDirectArenas=0, -Dlog4j.shu
tdownHookEnabled=false, -Dlog4j2.disable.jmx=true, -Dlog4j2.formatMsgNoLookups=true, -Djava.locale.providers=COMPAT, -Dcom.redhat.fips=false, -D
es.enforce.bootstrap.checks=true, -Xmx512m, -Xms512m, -XX:MaxDirectMemorySize=256m, -XX:+HeapDumpOnOutOfMemoryError, -Des.path.home=/opt/sonarqu
be/elasticsearch, -Des.path.conf=/opt/sonarqube/temp/conf/es, -Des.distribution.flavor=default, -Des.distribution.type=tar, -Des.bundled_jdk=fal
se]
sonarqube | 2026.04.29 17:28:17 INFO es[[o.e.p.PluginsService] loaded module [analysis-common]
sonarqube | 2026.04.29 17:28:17 INFO es[[o.e.p.PluginsService] loaded module [lang-painless]

```

Open dashboard:

```
http://localhost:9000
```

Login:

```
admin / admin
```

Step 2 — Create Sample Java App

Create project structure:

```
yuvraj_singh@LUCIF3R: /Lab9 sample-java-app$ tree ..
..
├── docker-compose.yml
└── sample-java-app
    ├── pom.xml
    └── src
        ├── main
        │   └── java
        │       └── com
        │           └── example
        │               └── Calculator.java
7 directories, 3 files
```

Create `Calculator.java`

```
yuvraj_singh@LUCIF3R: ~/ × + ▼
GNU nano 7.2 src/main/java/com/example/Calculator.java *
package com.example;

public class Calculator {

    // — BUG: Division by zero is not handled —————
    // If someone calls divide(5, 0), this will crash at runtime.
    public int divide(int a, int b) {
        return a / b;
    }

    // — CODE SMELL: Unused variable —————
    // The variable 'unused' is declared but never used.
    // SonarQube flags this as unnecessary clutter.
    public int add(int a, int b) {
        int result = a + b;
        int unused = 100; // ← code smell: delete this line
        return result;
    }

    // — VULNERABILITY: SQL Injection risk —————
    // Building a query by concatenating user input is dangerous.
    // An attacker could pass: "1 OR 1=1" and get all users.
    public String getUser(String userId) {
        String query = "SELECT * FROM users WHERE id = " + userId;
        return query;
    }

    // — CODE SMELL: Duplicated code —————
    // The two methods below do exactly the same thing.
    // This is copy-paste code and should be a single method.
    public int multiply(int a, int b) {
        int result = 0;
        for (int i = 0; i < b; i++) {
            result = result + a;
        }
        return result;
    }

    public int multiplyAlt(int a, int b) {
        int result = 0;
        for (int i = 0; i < b; i++) {
            result = result + a; // ← exact duplicate of multiply()
        }
        return result;
    }

    // — BUG: Null pointer risk —————
    // If 'name' is null, calling .toUpperCase() will throw
    // a NullPointerException at runtime.
    public String getName(String name) {
        return name.toUpperCase();
    }
}
```

Step 3 — Create Maven Config

Create `pom.xml`

```
yuvraj_singh@LUCIF3R: ~/ pom.xml *
GNU nano 7.2
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <!-- Project identity -->
  <groupId>com.example</groupId>
  <artifactId>sample-app</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>11</maven.compiler.source>
    <maven.compiler.target>11</maven.compiler.target>

    <!-- SonarQube connection settings -->
    <sonar.projectKey>sample-java-app</sonar.projectKey>
    <sonar.host.url>http://localhost:9000</sonar.host.url>
    <!-- Replace with your actual token (generated in Step 3) -->
    <sonar.login>YOUR_TOKEN_HERE</sonar.login>
  </properties>

  <dependencies>
    <!-- JUnit for unit tests -->
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <!-- This plugin lets us run: mvn sonar:sonar -->
      <plugin>
        <groupId>org.sonarsource.scanner.maven</groupId>
        <artifactId>sonar-maven-plugin</artifactId>
        <version>3.9.1.2184</version>
      </plugin>
    </plugins>
  </build>
</project>
```

Step 4 — Generate Token

Open dashboard → profile → **My Account** → **Security**

Generate token:

Name	Type	Project	Last use	Created	Expiration	
scanner-token	Global		Never	April 29, 2026	May 29, 2026	Revoke

Step 5 — Run Scan

```
cd sample-java-app
```

Run scanner:

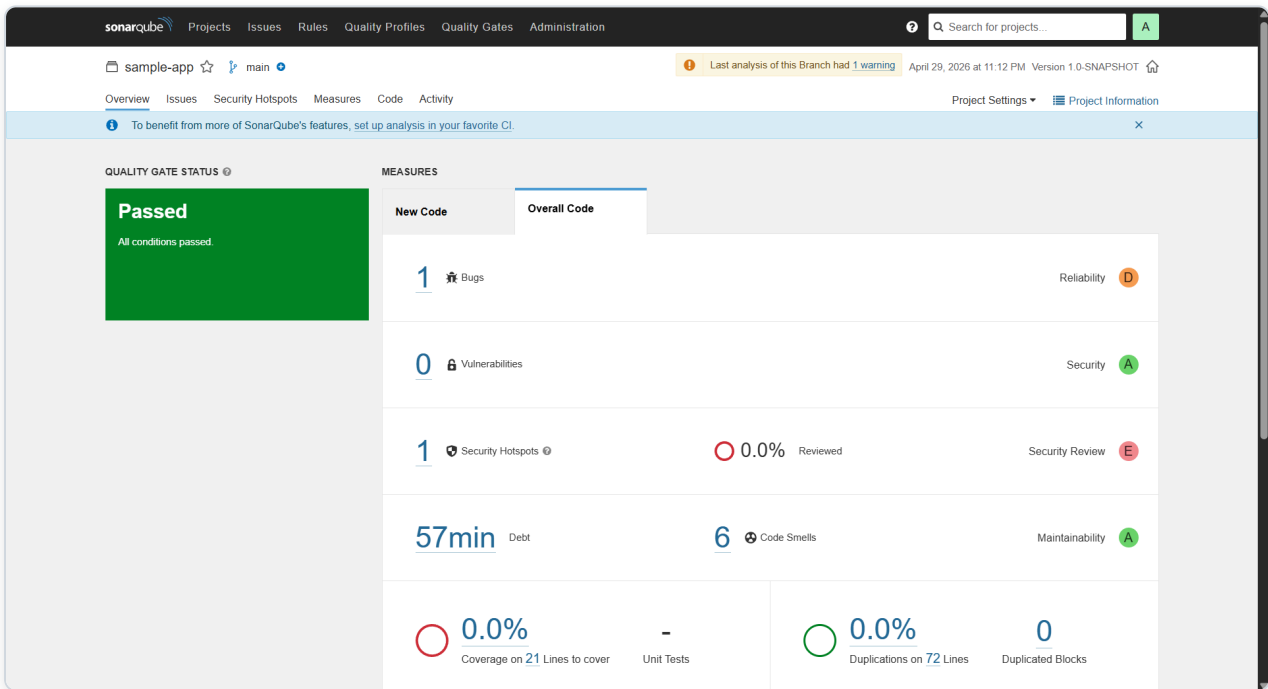
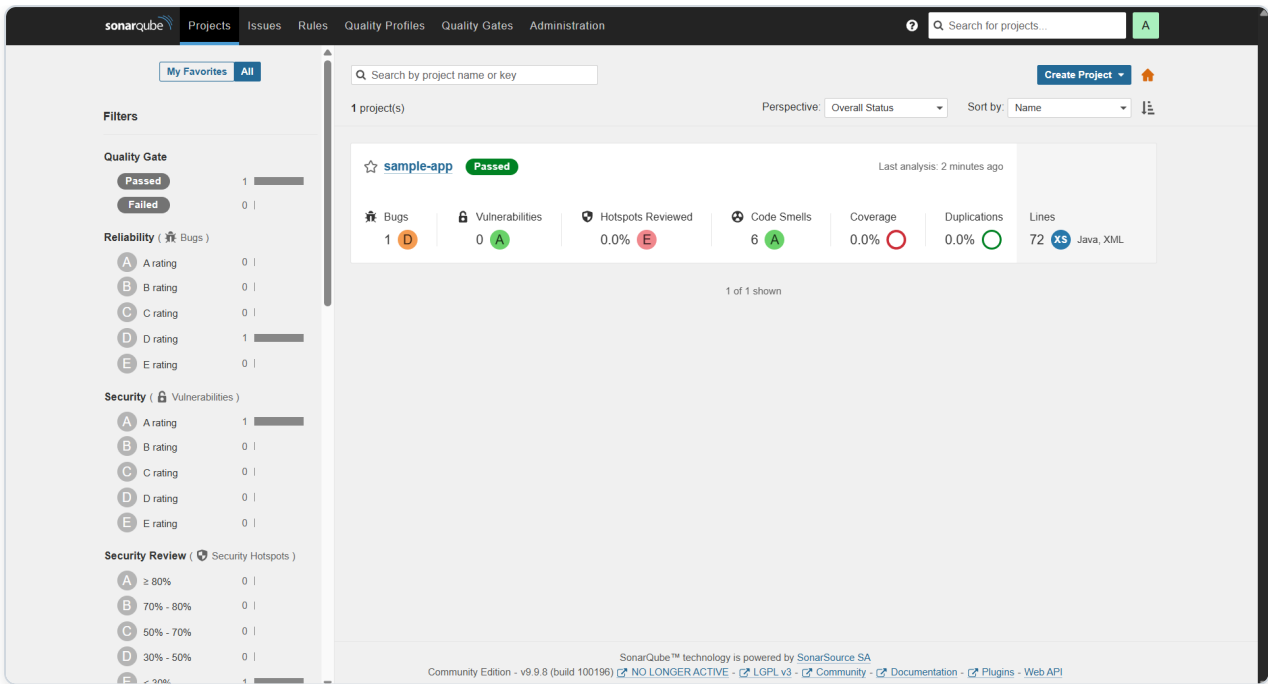
```
yuvraj_chamta@LUCIF3R: /Lab9/ sample-java-app? mvn clean verify sonar sonar
-Dsonar.projectKey=sample-app \
-Dsonar.host.url=http://localhost:9000 \
-Dsonar.login=sqa_aaf7e95b8b1e84b676f38590b88bd8fe963a877c
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:sample-app >-----
[INFO] Building sample-app 1.0-SNAPSHOT
[INFO] [ jar ]-----
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-clean-plugin/2.5/maven-clean-plugin-2.5.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-clean-plugin/2.5/maven-clean-plugin-2.5.pom (3.9 kB
at 11 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/22/maven-plugins-22.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/22/maven-plugins-22.pom (13 kB at 155 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/21/maven-parent-21.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/21/maven-parent-21.pom (26 kB at 347 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/apache/10/apache-10.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/apache/10/apache-10.pom (15 kB at 197 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-clean-plugin/2.5/maven-clean-plugin-2.5.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-clean-plugin/2.5/maven-clean-plugin-2.5.jar (25 kB
at 277 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-surefire-plugin/2.12.4/maven-surefire-plugin-2.12.
4.pom
[INFO] Analysis report generated in 36ms, dir size=130.9 kB
[INFO] Analysis report compressed in 9ms, zip size=20.3 kB
[INFO] Analysis report uploaded in 256ms
[INFO] ANALYSIS SUCCESSFUL, you can find the results at: http://localhost:9000/dashboard?id=sample-app
[INFO] Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
[INFO] More about the report processing at http://localhost:9000/api/ce/task?id=AZ3aVKh6iRUxODAhgPfg
[INFO] Analysis total time: 3.787 s
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 18.996 s
[INFO] Finished at: 2026-04-29T17:42:59Z
[INFO]
yuvraj_singh@LUCIF3R:~/Labg sample-java-appf |
```

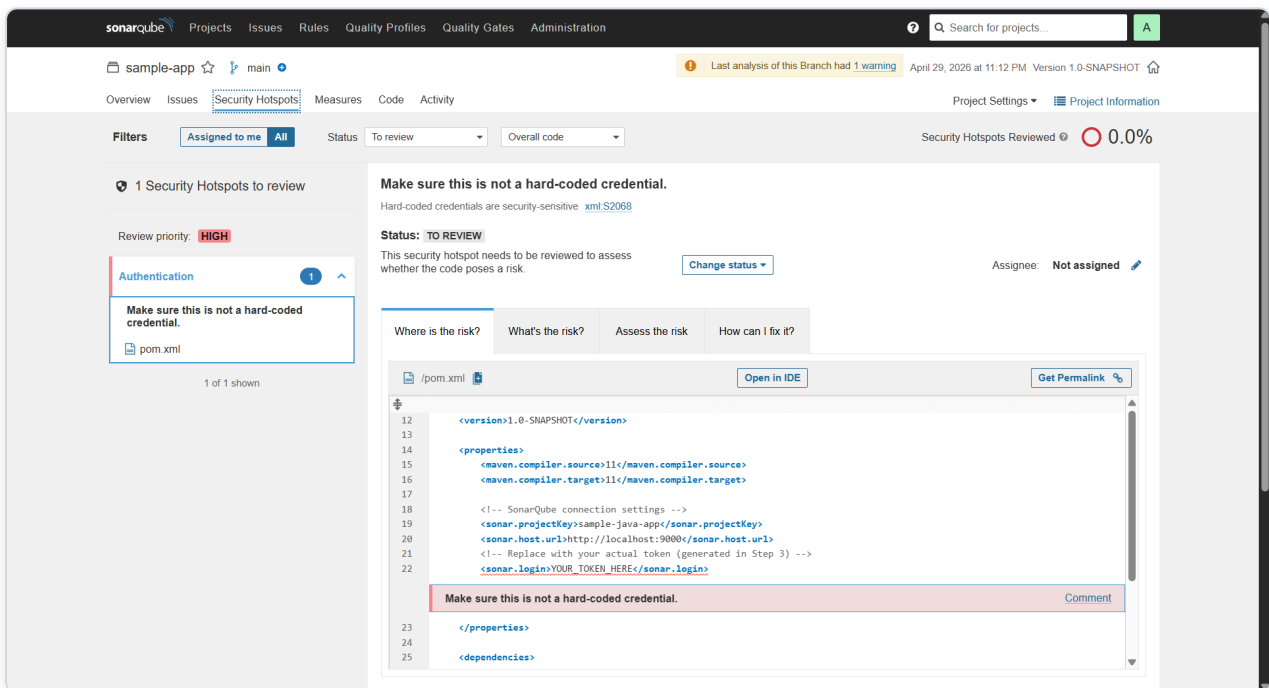
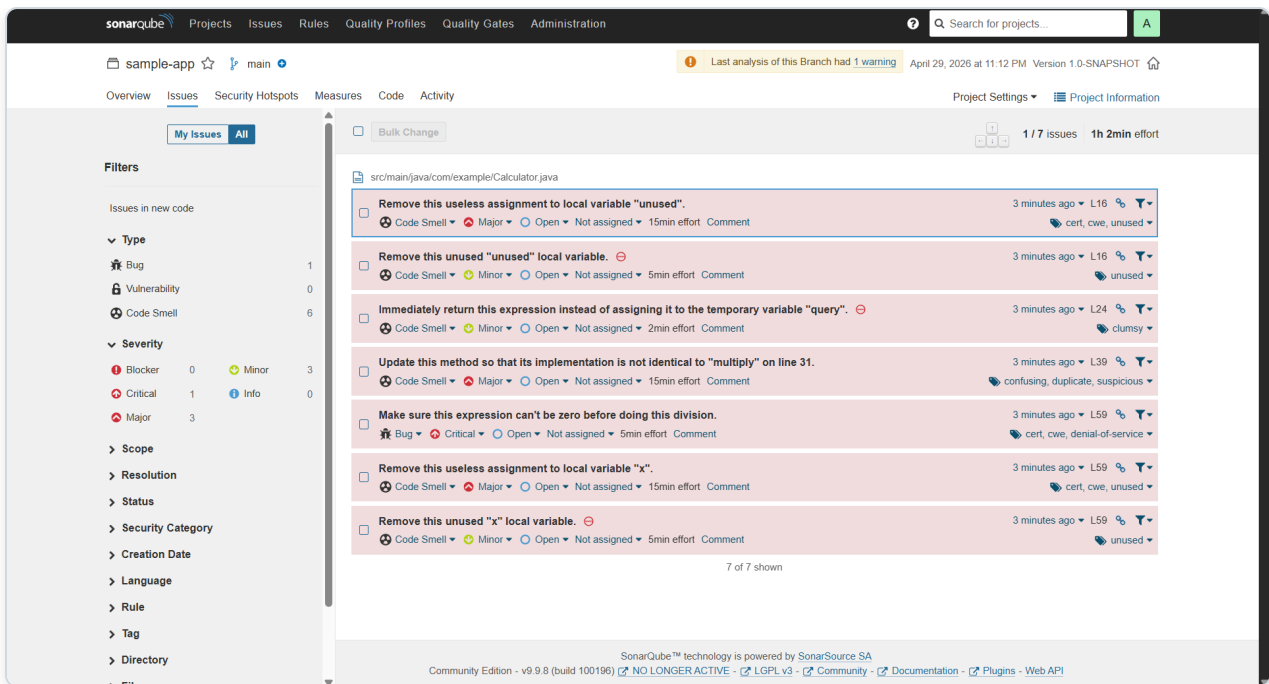
Process:

1. Maven compiles project
2. Scanner collects code
3. Sends analysis to server
4. Server evaluates rules
5. Results stored in PostgreSQL

Step 6 — View Results

Open:





You will see:

- Bugs
- Vulnerabilities
- Code Smells
- Duplication
- Technical Debt
- Quality Gate

Results

Detected issues include:

- Division by zero bug
 - SQL injection vulnerability
 - Unused variable
 - Duplicate methods
 - Null pointer risk
 - Empty catch block
-

Conclusion

In this experiment, SonarQube was deployed using Docker with PostgreSQL.

A Java project was analyzed using the Maven Sonar Scanner plugin, and issues such as bugs, vulnerabilities, and code smells were detected and visualized through the SonarQube dashboard.